# Agent Failure Boundaries: A First-Principles Security Taxonomy for Autonomous AI Systems

Aryan Haghighi

*Plarix Security Research*

*plarix.dev · security@plarix.dev*

20 March 2026

**Abstract**

We present a first-principles security taxonomy for autonomous AI agent systems organized around four invariant structural boundaries in the agentic execution loop. The taxonomy derives its categories not from a catalog of known attack techniques but from a minimal ontology of an AI agent: context, model, agent, and act. From this loop, four failure boundaries emerge: AFB01 Context Poisoning, AFB02 Model Boundary Compromise, AFB03 Instruction Hijack, and AFB04 Unauthorized Action. We argue these boundaries are structurally stable across single-agent, multi-agent, daemon, and hierarchical deployment configurations. Beyond the base taxonomy, this paper formalizes three additional contributions. First, the Canonical Execution Event (CEE): a four-field normalization schema providing the ground-truth record for policy evaluation and audit at every agent action. Second, temporal context entropy: a failure mode in which an agent accumulates dangerous information state through sequences of individually authorized operations. Third, a multi-agent trust propagation model showing that agent-to-agent communication introduces an attack surface that grows super-linearly with the number of agents and cannot be reduced to repeated single-agent boundary analysis. Two product architectures follow directly from the taxonomy: the Agent Policy Gateway (APG), targeting AFB04 as the decisive last-mile enforcement point, and the Selective Disclosure Layer (SDL), governing the perception side through controlled context exposure. Stress tests against six architectural variations confirm the four-boundary model remains complete at the ontological level as system complexity scales.

## 1. Introduction

Autonomous AI agents with access to real-world tools have moved from research prototypes into production deployments at substantial scale. Agents that read and write files, execute code, query databases, send messages, and call external APIs now operate in financial services, software engineering pipelines, customer-facing products, and enterprise automation. The transition from text generation to consequential action has happened faster than the security infrastructure required to govern it.

Security research on these systems has developed in parallel, but the dominant organizing principle remains enumeration: catalogs of named attack techniques such as prompt injection, jailbreaking, data leakage, model extraction, and excessive agency. Technique catalogs serve a purpose, but they share a structural

problem. They grow without bound as adversaries innovate. Categories mix levels of abstraction, making it difficult to reason about where an enforcement product should intervene. And they tend to describe the mechanisms of failure rather than the stable structural surfaces at which those failures can be intercepted.

This paper takes a different approach. Rather than starting from attack techniques, we start from the execution structure of the agent itself, asking: what is the minimal loop of an AI agent, and at which transitions in that loop can failure be introduced? From a four-element ontology, a small set of structural boundaries emerges. These boundary types are invariant: the specific mechanism of attack changes, but the transition type at which it operates does not. An enforcement architecture built around invariant boundary types does not become obsolete when novel attack variants appear.

This version of the taxonomy extends earlier work in three directions. We introduce the Canonical Execution Event as a formal normalization of every agent action that serves as the ground-truth input to policy enforcement. We identify and characterize temporal context entropy, a failure mode absent from single-boundary analysis. And we develop a formal treatment of multi-agent trust propagation, demonstrating that the inter-agent communication surface is qualitatively distinct from the sum of its single-agent parts.

### 1.1 Contributions

- A formally stated AFB taxonomy with precise boundary definitions grounded in the minimal agentic loop ontology.
- The Canonical Execution Event (CEE): a four-field normalization schema providing the atomic unit for policy enforcement and audit.
- Temporal Context Entropy: formalization of a failure mode in which legitimate authorized operations accumulate dangerous agent information state over time.
- A multi-agent trust propagation model showing that agent-to-agent communication channels produce an attack surface scaling as $O(N^2)$ and require dedicated attestation mechanisms.
- Stress tests of the four-boundary model across six architectural variations including heterogeneous models, no-code frameworks, long-running daemons, and hierarchical swarms.
- Two product wedge specifications, APG and SDL, derived directly from the boundary model without appeal to market analysis.

## 2. Related Work

***Adversarial Machine Learning.***

Adversarial ML research [Goodfellow et al., 2014; Carlini and Wagner, 2017] addresses attacks on models treated as static input-output functions: perturbations, model inversion, membership inference, and extraction. This body of work is foundational but largely pre-agentic. It assumes a fixed attack surface defined by the model's input space. Agent systems break this assumption by introducing feedback loops, persistent memory, tool access, and consequential action in the world.

***LLM Application Security.***

The OWASP Top 10 for LLM Applications [OWASP, 2025] provides the most widely cited taxonomy of LLM-specific risks: prompt injection (LLM01), sensitive information disclosure (LLM02), supply chain (LLM03), data and model poisoning (LLM04), improper output handling (LLM05), and excessive agency (LLM06). The OWASP Agentic Security Initiative [OWASP ASI, 2026] extends these to agent-specific failure modes including agent goal hijack (ASI01), tool misuse (ASI02), identity and privilege abuse (ASI03), and memory and context poisoning (ASI06). Section 12 provides an explicit AFB-to-OWASP mapping. The AFB model does not replace these categories; it compresses them into a smaller set of structural boundaries that are more directly actionable for enforcement architecture.

***Policy and Access Control.***

NIST SP 800-63C [NIST, 2017] introduces selective disclosure, the principle that systems should receive the minimum necessary attribute representation for a given task. This principle informs the SDL wedge in Section 11. Attribute-based access control frameworks [Hu et al., 2014] provide relevant precedent for policy specification at the resource-operation level, though existing ABAC frameworks do not address the dynamic, context-dependent action spaces that characterize agent systems.

## 3. Minimal Ontology of an AI Agent

Security architecture requires terminological precision. Categories that are poorly defined produce enforcement boundaries that are poorly placed. We begin with the smallest set of concepts sufficient to describe the agentic execution loop at the abstraction level relevant to security design. Four elements are necessary and sufficient.

### 3.1 Context

Context is any information available to the system before the next step is chosen. It includes system prompts, user inputs, retrieved documents, database records, prior conversation, tool outputs, environment observations, memory contents, and state variables. The key property is not the source but the visibility: the system can read it and be shaped by it.

> **Initial context** is the starting seed: role specification, task description, system prompt, initial data, and operational constraints present before the first execution step.
>
> **Ongoing context** is the evolving state during execution: intermediate outputs, errors, prior acts, tool responses, environment changes, and consequences of previous steps.

This distinction carries operational weight. Many attacks do not corrupt the initial prompt; they corrupt the rolling informational state the agent relies on mid-execution. Consequence spoofing, presenting the agent with a false record of what its prior action produced, is an ongoing context attack, not an initial prompt attack. Both belong to the same boundary regardless of delivery mechanism.

### 3.2 Model

The model is the token-processing reasoning engine. At this abstraction level it receives input tokens and produces output tokens. Internal chain-of-thought, decoding strategies, provider-side orchestration, and hidden reasoning mechanisms are treated as internal to the model box. The point is to mark the model as the

component where visible context is transformed into outputs that will subsequently shape action: context in, model output out.

### 3.3 Agent

The agent is the action-enabling machinery around the model: orchestration logic, planners, workflows, memory management, tool interfaces, API clients, MCP clients and servers, retrieval components, state machines, and execution scaffolding. If the model is the reasoning core, the agent is the machinery that converts reasoning output into operations. The model emits intent-like output; the agent makes that output operable.

### 3.4 Act

An act is any operation the AI agent chooses or attempts. The definition is intentionally broad: reading, writing, selecting a tool, changing a plan, calling an API, generating code, modifying a file, sending a message, retrying after failure, updating memory. An act is not restricted to visibly destructive operations. Any chosen operation that advances the execution loop is an act, because security policy must govern the full action space, not only its destructive subset.

### 3.5 Consequence

A consequence is what is returned from an act and fed back into ongoing context. It may include world-state changes, tool responses, error messages, partial failures, or subtle state shifts. Because consequence feeds ongoing context, the agent operates on a world shaped by its own prior acts. This feedback property has direct implications for the temporal failure mode discussed in Section 6.

## 4. The Minimal Agentic Loop

With the four ontological elements fixed, the minimal execution loop takes the following form:

```
Context → AI Agent → Act → Consequence → Context
```

Expanded with the full ontology:

```
(Initial context + Ongoing context) → Model → Agent → Act → Consequence
→ New ongoing context
```

The value of this representation is its compactness. Once the loop is stated explicitly, one can pose a precise security question: at which transitions in this loop can corruption, misdirection, or unauthorized behavior be introduced? The answer is a short list of structural transitions, not an unbounded catalog of technique variants.

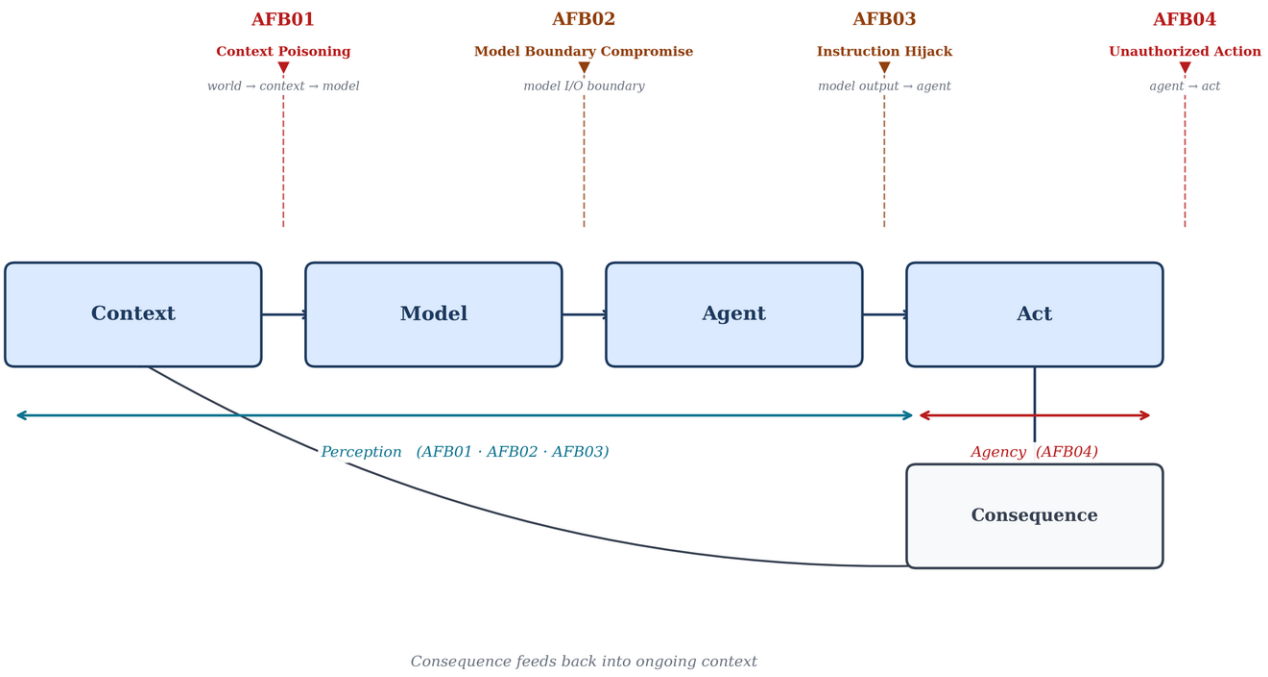


*Figure 1. The minimal agentic loop with the four Agent Failure Boundaries (AFBs) positioned at their respective transitions. AFB01 and AFB02 operate on the perception side; AFB03 sits at the model-to-agent transition; AFB04 is the last-mile enforcement point before the agent touches world state.*

The loop is not a single-pass pipeline. In long-running agents, it executes continuously. Each iteration's consequence modifies the context for the next, producing a temporal accumulation dynamic that single-pass analysis does not capture. Section 6 addresses this property directly.

## 5. The Four Agent Failure Boundaries

We define four Agent Failure Boundaries (AFBs). Each corresponds to a critical transition in the minimal agentic loop, marking an interface where information or control crosses from one component to another and can be corrupted, redirected, or abused. The boundaries are not arbitrary: they follow from the four transitions that the loop ontology requires.

### 5.1 AFB01: Context Poisoning

**Transition:** External world / tool outputs → Context → Model

AFB01 describes the failure condition in which the model ingests context that has been corrupted, forged, or manipulated, and subsequently produces outputs that reflect the compromise. The critical abstraction is that the delivery mechanism is secondary. Whether the poisoned content arrived through a prompt, a retrieved document, a vector store entry, a tool response, or a memory record, the failure type is the same: the model treats content it should not trust as part of its informational basis for the next step.

Specific failure instances include:

- Direct and indirect prompt injection via user input or external content.
- Hidden instructions in retrieved documents, emails, or web pages.
- Poisoned vector store entries that corrupt retrieval-augmented generation.
- Forged tool responses misrepresenting the outcome of a prior act.
- Consequence spoofing: false feedback about the result of a previous operation.
- Compromised memory records that redirect agent goals across sessions.

### 5.2 AFB02: Model Boundary Compromise

**Transition:** Model I/O boundary (input and output surfaces)

AFB02 marks failures at the model-provider boundary. The concern is integrity, confidentiality, and trust at the surface where information enters or exits the model. The initial shorthand of API safety was rejected as too narrow: the boundary encompasses transport security, provider-side handling, middleware layers, and model-serving infrastructure.

AFB02 failure instances include:

- Prompt leakage: system prompt or user context exposure to unauthorized parties.
- Output leakage: sensitive information in model responses accessible beyond the intended recipient.
- Tampering in transit: modification of inputs or outputs between agent and model provider.
- Unauthorized middleware: intermediary layers that alter prompts or responses without authorization.
- Provider-side mishandling: serving-layer failures that expose or corrupt information.

AFB02 matters because a secure architecture cannot assume the model boundary is automatically trustworthy. In layered architectures with model brokers, routing layers, or multi-provider configurations, the model I/O surface multiplies and each instance is a distinct AFB02 exposure.

### 5.3 AFB03: Instruction Hijack

**Transition:** Model output → Agent

AFB03 sits at the boundary between model output and the agent layer that operationalizes it. The failure occurs when the model produces unsafe, distorted, or unauthorized instructions that the agent then executes. Two distinct pathways converge at this boundary:

**Adversarial pathway.** A malicious upstream input, via AFB01 or AFB02, has redirected model output toward unsafe instructions. The agent receives instructions it should never have been handed.

**Endogenous pathway.** The model, without adversarial influence, produces unsafe instructions due to hallucination, misgeneralization, or reasoning failure. No external attacker is required.

The two pathways produce the same effect: the instruction channel from model to agent is no longer reliable. The term instruction hijack captures both, since in both cases the agent receives instructions that distort or exceed what was authorized by the operator.

*AFB03 functions as a detection boundary rather than a standalone enforcement boundary. A reasoning layer operating at AFB03 classifies the integrity of model output; enforcement occurs downstream at*

*AFB04. The CEE architecture in Section 9 formalizes this relationship.*

### 5.4 AFB04: Unauthorized Action

**Transition:** Agent → Act

AFB04 is the decisive last-mile boundary. The failure occurs when the agent attempts or performs an act it is not authorized to perform, should not perform in the current context, or performs in a manner that violates defined policy, regardless of why the agent arrived at that decision. AFB04 is where excessive agency becomes concrete: the preceding boundaries shape what the system believes and intends; AFB04 is where that intent becomes world-state change.

Two sub-properties are analytically distinct but both locate at AFB04:

**Permission.** What authority, access, and allowable operations the agent possesses given its identity, context, and current policy state.

**Execution.** What operation the agent is actually attempting or performing.

A core architectural insight: an agent that has been corrupted upstream at AFB01, AFB02, or AFB03 can still fail safely if AFB04 is properly enforced. Upstream compromise gains a route into the world only when AFB04 is absent or underspecified. This property motivates the APG as the first product wedge.

## 6. Temporal Context Entropy

The four-boundary model treats the agentic loop as structurally uniform across iterations: the same boundaries apply at each step. This is correct at the ontological level. It does not, however, capture a failure mode specific to long-running autonomous agents: temporal context entropy.

Temporal context entropy refers to the progressive accumulation of dangerous information state through legitimate, authorized operations over time. No individual act is unauthorized. No individual context entry is poisoned. Nevertheless, the agent's accumulated context at step N represents a security state qualitatively different from its context at step 1.

> *Consider an agent authorized to read configuration files and write summaries. At step 1, it reads a single low-sensitivity file, well within policy. By step 50, it has read configuration files, log outputs, user records, and API response bodies across the system. Its context now contains a comprehensive model of internal architecture, sensitive endpoint paths, and credential patterns. Step 51 is another authorized read. But the agent at step 51 is not the same security entity as the agent at step 1, despite having performed only authorized operations throughout.*

This failure mode does not reduce to any single AFB. It is not AFB01 because the context is not poisoned; it is legitimately acquired. It is not AFB04 because no individual act is unauthorized. It is an emergent property of the temporal loop structure that point-in-time boundary analysis misses.

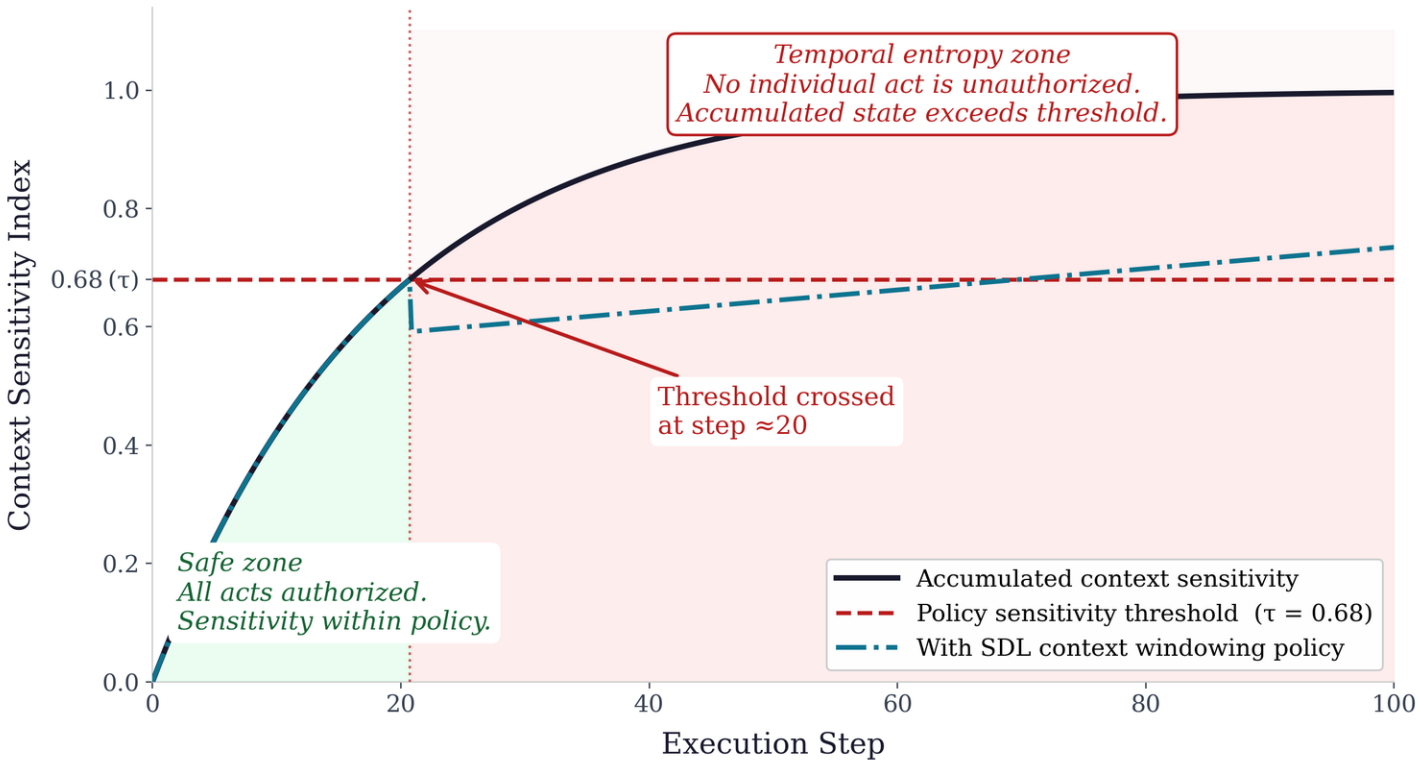


*Figure 2. Illustration of temporal context entropy. Even when every individual act is authorized, accumulated context sensitivity can exceed a defined policy threshold (dashed line) after sufficient execution steps. The inflection point depends on the agent's access breadth and session length.*

### 6.1 Architectural Implications

Addressing temporal context entropy requires controls that operate across the loop's temporal dimension rather than at individual boundary instances. Two mechanisms are relevant:

> **Context accumulation monitoring.** A layer that evaluates the agent's cumulative information state at defined intervals, flagging when aggregate sensitivity exceeds a policy threshold. The unit of concern is the accumulated set, not any individual entry.
>
> **Context windowing policy.** Constraints that limit how much information an agent retains across act cycles, forcing periodic reduction to a defined minimum sufficient representation. This is the temporal analog of the SDL's spatial minimization principle.

Measuring context sensitivity is a non-trivial problem. The threshold in Figure 2 is illustrative; in practice it requires domain-specific calibration and a sensitivity metric for heterogeneous accumulated content. This remains an open problem.

## 7. Completeness of the Four-Boundary Model

A taxonomy should not proliferate categories beyond structural necessity. A fifth AFB would be warranted only if a new invariant transition type could be identified that cannot be represented as a subtype, repetition,

or composition of the existing four. We tested several candidate boundaries against this criterion.

> **Feedback spoofing:** presenting the agent with false output from a prior act. Collapses to AFB01, via the consequence-to-ongoing-context channel.
>
> **Cross-session memory poisoning:** a persistent memory record redirecting future agent goals. Collapses to AFB01, applied to a future iteration's context.
>
> **Tool response manipulation:** a malicious tool response altering agent behavior. Collapses to AFB01, via the tool output channel.
>
> **Multi-agent message manipulation:** an agent's output corrupting another agent's context. Collapses to AFB01 at the receiving agent, or AFB03 if the compromise operates at the instruction level.
>
> **Authorization escalation:** an agent acquiring and exercising permissions beyond its defined scope. Collapses to AFB04.

Temporal context entropy, despite being a distinct failure phenomenon, does not require a fifth boundary type. It is a property of the loop's temporal dimension, an emergent behavior of repeated legitimate boundary crossings, not a new transition type.

> *The four-boundary model is complete at the level of transition types within the minimal agentic loop. Scale and complexity increase the number of AFB instances without introducing new AFB types. This distinction matters: each agent and subagent in a multi-agent system carries its own instance of all four boundaries and therefore requires its own enforcement. The boundary type is the same. The enforcement obligation is not shared — it is per-instance. And the inter-agent channels that connect those instances create a surface that per-agent enforcement does not cover. That surface is the subject of Section 8.*

## 8. Multi-Agent Systems and Trust Propagation

Single-agent security analysis treats trust as a directed chain: human operator to system prompt to agent. The agent is a leaf node; trust flows in one direction and its origin is always traceable to human authorization. Multi-agent architectures break this model in two ways that must be distinguished carefully.

The first is quantitative. Every agent and subagent in a multi-agent system carries its own instance of all four Agent Failure Boundaries. A system of N agents does not have four boundaries; it has 4N boundary instances. Each one requires its own enforcement. Enforcement at the orchestrator level does not cover boundary failures at worker agents. Each agent must be treated as an independent enforcement unit.

The second is qualitative. The connections between those per-agent enforcement instances — the inter-agent communication channels — create an attack surface that does not exist in single-agent systems and is not addressed by per-agent enforcement alone. An orchestrator receives output from a worker agent and, absent an attestation mechanism, has no principled basis for determining whether that output should be trusted. The worker may have been compromised at its own AFB01. Its output is consistent with the poisoned context it received. The orchestrator ingests this output as trusted input, propagating the compromise without any boundary failure detectable at the receiving end. This is trust laundering through apparently authorized channels, and it is the qualitative difference between single-agent and multi-agent security.

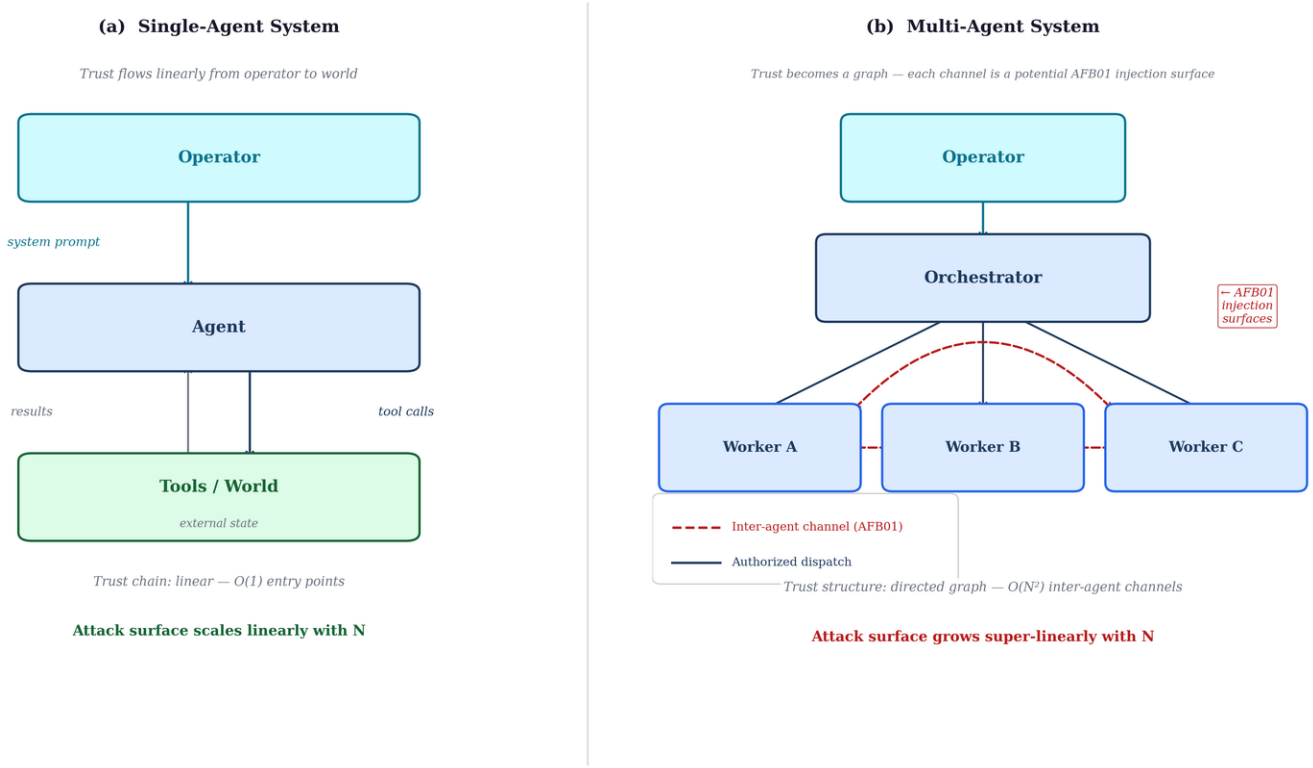


*Figure 3. Single-agent versus multi-agent trust structure. In the single-agent case, trust flows linearly from operator to agent to tools. In the multi-agent case, agent-to-agent communication channels scale as $O(N^2)$ in the fully connected configuration, and each channel is a potential AFB01 injection surface that bypasses human-origin validation.*

### 8.1 The Trust Propagation Problem

The multi-agent attack surface is qualitatively distinct from repeated single-agent boundary analysis in two respects:

> **Exponential surface.** In a system with N agents, agent-to-agent communication channels scale as $O(N^2)$ in the fully connected case. Each channel is a potential AFB01 injection surface that bypasses human-origin validation. The attack surface grows super-linearly with the number of agents.
>
> **Trust laundering.** An adversary who compromises a low-privilege worker can use that agent's output to inject instructions into a high-privilege orchestrator's context, effectively elevating the compromise through an apparently authorized inter-agent channel.

### 8.2 Agent Identity and Attestation

Addressing trust propagation requires a mechanism absent from current agent frameworks: agent identity attestation. Before one agent's output becomes another agent's trusted context, the receiving agent requires a basis for verifying:

- That the sending agent was operating within its defined policy scope.
- That the sending agent's context was not compromised at its own AFB01.

- What trust level the message carries, specifically what authority the sender was operating under.
- That the message was not modified in transit, an AFB02 property.

The architectural analog is TLS for inter-service communication: it does not prevent a compromised service from sending malicious data, but it provides authenticity and integrity guarantees for the channel itself. Agent attestation extends this by additionally conveying the policy scope under which the sender operated, giving the receiver a basis for trust-level assignment. This remains an open design problem.

## 9. The Canonical Execution Event

Policy enforcement requires a normalized representation of what is being evaluated. Ad hoc representations, raw tool call objects, free-form log strings, and framework-specific event schemas, produce enforcement logic that is brittle across agent frameworks and deployment configurations. We define the Canonical Execution Event (CEE) as the normalized ground-truth record of every agent action attempt.

The CEE has exactly four fields:

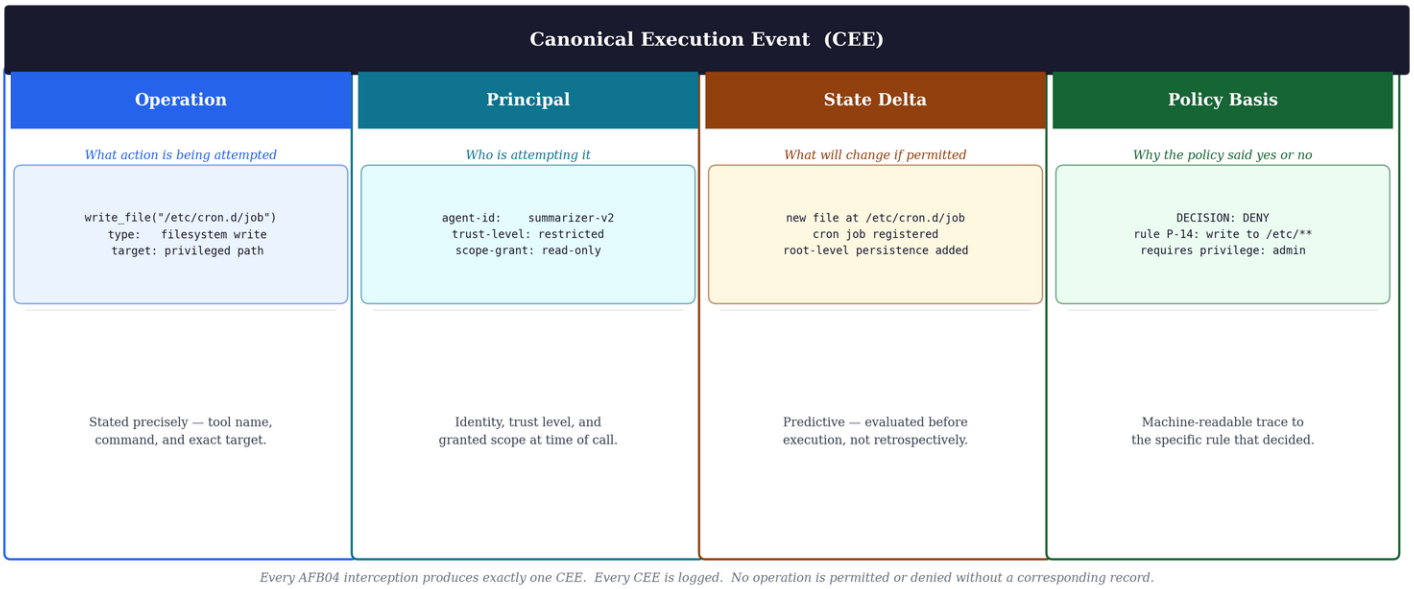


*Figure 4. The Canonical Execution Event (CEE) schema. Every AFB04 interception produces exactly one CEE. Every CEE is logged. The four fields together constitute the complete record required for policy evaluation, audit, and observability.*

**Operation.** The action being attempted, stated precisely. Not "file access" but "write to /etc/credentials". Not "send message" but "send email to external domain". Precision in the operation field is what makes policy rules specific enough to be enforceable.

**Principal.** The identity of the entity attempting the operation: the specific agent, subagent, service, or delegated actor. Principal carries the context required for identity-based policy evaluation: what scope this agent was granted, under whose authorization it operates, and what trust level it holds.

**State Delta.** What will change in the world if the operation is permitted to execute. State delta makes the real-world consequence of the operation explicit at the point of policy evaluation, before execution

occurs. This field is predictive, not retrospective.

**Policy Basis.** Why the policy said yes or no. The specific rule, condition, or reason that determined the decision. Policy basis makes enforcement auditable: every decision traces to a specific policy element, and the trace is machine-readable.

Every interception at AFB04 produces exactly one CEE. Every CEE is logged. No operation is permitted or denied without a corresponding CEE record. The CEE log is the authoritative record of what the agent system did and the policy basis for each decision.

### 9.1 CEE and the AFB03 Detection Relationship

The CEE is generated at AFB04, the enforcement boundary. The Policy Basis field, however, can incorporate signal from upstream boundary analysis, including AFB03 classification. When a reasoning layer at AFB03 classifies model output as potentially hijacked, that classification is available to the AFB04 policy evaluation as an input to the Policy Basis decision. This resolves the enforcement mechanism question at AFB03: AFB03 does not require its own enforcement actuator. It feeds a risk classification into the CEE, where enforcement occurs. The architecture is:

```
AFB03 detection → risk classification → enriches CEE Policy Basis →
AFB04 enforcement decision
```

## 10. Perception and Agency

Once the four-boundary model is in place, a second abstraction emerges at a higher level of product legibility. An AI agent has two major sides: what it sees, and what it can do.

**Perception** is the informational side of the system: the data, instructions, context, and operational cues the agent receives before acting. AFB01, AFB02, and AFB03 all operate on the perception side, governing what reaches the agent and in what form.

**Agency** is the operative side: what the agent can do. AFB04 is the agency boundary, where permission and execution become real and where intent becomes world-state change.

The relationship between perception and agency is asymmetric in a security-relevant way. Failures on the perception side become consequential only when they survive to AFB04 and produce an unauthorized act. A sufficiently strong AFB04 enforcement layer contains the blast radius of perception-side failures without eliminating them. This asymmetry motivates the ordering of product wedges: AFB04 enforcement first, perception controls second.

## 11. Product Wedges

The boundary model directly generates two product specifications. These are not derived from feature requests; they are the natural enforcement architectures that the boundary analysis requires.

### 11.1 Agent Policy Gateway (APG)

The Agent Policy Gateway is the enforcement mechanism at AFB04. Its function is to evaluate every intended agent act against defined policy and produce a binding allow-or-deny decision before the act touches world state. The APG consumes a CEE and evaluates it against:

- The specific operation being requested.
- The identity and trust level of the principal.
- The target resource or endpoint.
- The predicted state delta.
- The current context and execution history of the agent.
- Any upstream AFB03 risk classification from the detection layer.

In stronger enforcement configurations, the APG may additionally: require human approval for high-risk operations, downgrade a requested operation to a safer variant, strip dangerous parameters from an otherwise permitted call, or substitute a read for a requested write pending additional authorization.

> *The essential design principle: if an AI agent is going to touch state, it should not do so through unbounded trust in model output. It should cross a policy gate. The APG is framework-agnostic by construction. Its interception surface is the agent-to-act boundary, which exists regardless of which framework built the agent, which model powers its reasoning, or what deployment topology hosts it.*

### 11.1.1 Policy Language

The APG enforces policy, but the architecture of the APG is only as strong as the language in which policies are expressed. This is not a detail; it is the core usability question. Four approaches exist and each carries a distinct tradeoff.

**Declarative rule sets** specify explicit allow/deny conditions per operation, resource, and principal. They are auditable, composable, and require no runtime inference. Their limitation is expressiveness: they handle known operation types well and novel situations poorly. As agent action spaces expand, declarative rules require constant extension.

**Capability tokens** grant agents unforgeable authority over specific operations at initialization. The security model is clean and the runtime overhead is minimal. The limitation is that complex conditional policies — permit this operation only if the prior three operations produced these outcomes — are difficult to express in capability form.

**Natural language policies** interpreted by a reasoning model offer maximum expressiveness. They introduce a recursive trust problem: the system uses AI to enforce AI security. A policy interpreter that can be manipulated at AFB01 undermines the enforcement layer it is meant to provide.

**Formal attribute-based policies** in the tradition of XACML [Hu et al., 2014] provide the strongest logical guarantees but impose adoption costs that make them impractical outside highly regulated environments.

> *The v1 APG position is declarative rules as the primary layer, with natural language policy interpretation available as an explicitly bounded escape hatch for complex conditional cases. The escape hatch accepts the recursive trust risk but constrains it: the policy interpreter operates on a narrow, read-only view of the CEE and produces only a binary allow/deny signal with a mandatory policy basis string. It cannot modify the CEE, cannot access agent memory, and cannot initiate actions.*

> *Constraining the interpreter's capability surface limits the blast radius of a compromised interpreter without eliminating the expressiveness benefit.*

### 11.2 Selective Disclosure Layer (SDL)

The Selective Disclosure Layer governs the perception side, specifically what information the agent is permitted to see before it acts. The SDL addresses a structural vulnerability in default deployments: agents routinely receive raw sensitive context that exceeds the minimum representation necessary for their task. The core SDL principle, drawn from NIST SP 800-63C: the model should receive the minimum necessary representation for the task at hand, not the maximum available.

SDL mechanisms include:

- Tokenization: replacing sensitive identifiers with opaque tokens that preserve referential integrity without exposing raw values.
- Pseudonymization: replacing identifying attributes with derived values that support task completion without enabling re-identification.
- Attribute derivation: exposing derived properties rather than raw values where the task requires the property, not the underlying datum.
- Conditional disclosure: granting field access only when a policy condition is met, with revocation when the condition lapses.
- Layered visibility: components of a multi-model architecture receive different disclosure levels appropriate to their trust scope.

SDL faces a structural tension that must be stated explicitly rather than left implicit. LLMs require semantic content to reason. Tokenization that replaces a name with an opaque identifier works when the task does not require reasoning about that name in context. It breaks when the agent's task depends on the identity, role, relationship, or history associated with the suppressed value. A tokenized patient record supports a billing classification task. It cannot support a clinical reasoning task that requires knowing the patient's prior diagnoses.

This is the SDL generality-utility tension. A fully general SDL — one that suppresses all sensitive fields uniformly — achieves strong protection at the cost of making agents unable to complete tasks that require the suppressed information. A fully task-aware SDL — one that releases exactly what each task requires — approaches maximum utility but requires per-task disclosure policies that undermine the set-and-forget operational model.

> *The v1 position for SDL is therefore bounded and explicit: SDL governs structured sensitive fields with well-defined sensitivity classifications, applies tokenization and pseudonymization to identifiers where referential integrity is sufficient for the task, and uses conditional disclosure for fields where semantic content is required. Free-text sensitive data, where semantic content and sensitive content are inseparable, is out of scope for v1 SDL and is treated as an open research problem. This is not a weakness to be concealed; it is a boundary that makes the v1 scope honest and the v1 implementation feasible.*

SDL is not a replacement for APG. The two wedges are complementary: SDL reduces the informational surface available to the agent, limiting AFB01 exposure and reducing the value of AFB02 leakage; APG

enforces what the agent can do with the information it has received. SDL also has a temporal dimension that intersects with context entropy: the minimum necessary representation at step 1 may differ from the minimum necessary at step N, and SDL policy must account for accumulation dynamics, not only point-in-time disclosure.

## 12. Mapping to OWASP

The AFB taxonomy was developed independently of OWASP nomenclature but maps onto established OWASP categories in ways that confirm it is not detached from the existing risk landscape. The mapping claims overlap, not equivalence, and is interpretive rather than official. Its purpose is to show that the AFB model compresses several established risk categories into a smaller set of structural boundaries more directly actionable for enforcement architecture design.

| AFB | Description | OWASP LLM | OWASP Agentic |
|---|---|---|---|
| **AFB01 Context Poisoning** | *Model ingests corrupted context* | LLM01 Prompt Injection<br>LLM04 Data Poisoning<br>LLM08 Embedding Weaknesses | ASI06 Memory & Context Poisoning<br>ASI01 Agent Goal Hijack |
| **AFB02 Model Boundary Compromise** | *I/O boundary tampered or exposed* | LLM02 Info Disclosure<br>LLM03 Supply Chain | ASI04 Supply Chain |
| **AFB03 Instruction Hijack** | *Model output becomes unsafe instructions* | LLM01 Prompt Injection<br>LLM05 Output Handling | ASI01 Agent Goal Hijack<br>ASI02 Tool Misuse |
| **AFB04 Unauthorized Action** | *Agent executes unpermitted acts* | LLM05 Output Handling<br>LLM06 Excessive Agency | ASI02 Tool Misuse<br>ASI03 Privilege Abuse<br>ASI05 Code Execution |

*Table 1. AFB-to-OWASP mapping. Overlap is interpretive, not official.*

## 13. Stress Tests

A taxonomy that holds only for simple single-step agents is a diagram, not a framework. We stress-test the four-boundary model against six architectural variations and report whether the boundaries remain stable or whether additional boundary types are required.

### 13.1 Heterogeneous Model Configurations

Whether the reasoning engine is a hosted frontier model, a locally-run open-weight model, or a specialized fine-tuned system, the loop still requires context entering reasoning, a model boundary, output crossing to an agent layer, and acts crossing to world state. Model substitution changes implementation detail, not boundary type. **Result: boundaries stable.**

### 13.2 Heterogeneous Agent Frameworks

LangGraph, LlamaIndex, CrewAI, AutoGen, custom orchestrators, no-code workflow platforms, and MCP-native agents differ substantially in internal wiring. The AFB boundaries remain structurally identical across all of them. **Result: boundaries stable.**

### 13.3 Long-Running Daemon Systems

Continuously executing agents produce more ongoing context and more repeated acts per session. The loop executes thousands of times rather than once. Temporal context entropy (Section 6) emerges as a significant additional consideration. No new boundary type is required. **Result: boundaries stable; temporal effects require separate treatment.**

### 13.4 Multi-Agent and Hierarchical Systems

In orchestrator-worker systems and hierarchical swarms, the same four boundaries apply at every agent. They do not disappear; they multiply. Each agent-to-agent communication channel is a potential AFB01 injection at the receiving agent. Trust propagation (Section 8) introduces a qualitatively distinct problem that single-agent analysis does not fully capture. **Result: boundary types stable; trust propagation is an additional architectural concern, not a new boundary type.**

### 13.5 Boundary Merging in Concrete Implementations

Tightly-fused local runtimes may make the model and agent components nearly indistinguishable in code. A direct function-calling stack may compress the visible gap between model output and agent execution. The transition still occurs; it is simply implemented at a finer granularity. **Result: boundaries stable at the conceptual level.**

### 13.6 Computer-Use and Embodied Agents

Agents controlling browsers, desktop applications, and GUI environments have action spaces that are difficult to enumerate precisely. AFB04 enforcement in these contexts is harder to specify, but the boundary type is the same. This is a policy specification challenge, not a taxonomy challenge. **Result: boundaries stable; policy completeness is a separate open problem.**

## 14. Limitations and Open Problems

The AFB model and the product wedges derived from it carry several limitations that constitute an explicit research agenda.

**Policy language completeness.** The v1 APG position of declarative rules with a bounded natural language escape hatch (Section 11.1.1) is a practical starting point, not a complete solution. Declarative rule sets require extension as agent action spaces grow. The natural language escape hatch carries a constrained but real recursive trust risk. A complete policy language for agentic systems — one that is expressive enough for complex conditional cases, auditable enough for enterprise compliance, and simple enough for developers to adopt without security expertise — does not yet exist.

**SDL semantic compression.** The generality-utility tension in SDL is real and bounded by the v1 scope defined in Section 11.2: structured fields with well-defined sensitivity classifications are in

scope; free-text sensitive data where semantic content and sensitive content are inseparable is explicitly out of scope. The open research problem is finding minimum-sufficient semantic representations for arbitrary content types, particularly in domains where sensitive information is embedded in natural language rather than structured fields.

**Temporal context entropy measurement.** The identification of temporal context entropy as a failure mode does not provide a measurement framework. What constitutes dangerous information density in accumulated context? How is a threshold defined without domain-specific calibration? These questions are open.

**Multi-agent attestation protocols.** The trust propagation problem in multi-agent systems requires attestation mechanisms that do not currently exist in standardized form. The design space for agent identity and policy-scope attestation is largely unexplored.

**Reasoning layer recursive trust.** An architecture that uses reasoning models at security boundaries introduces a recursive question: who validates the validator? A security reasoning model is itself an LLM and potentially subject to AFB01 through AFB04. Mitigations exist, including model diversity, narrow-scoped fine-tuning, and uncorrelated failure modes, but a complete treatment of recursive security architecture trust is an open problem.

## 15. Conclusion

We have presented a revised and extended Agent Failure Boundary taxonomy for autonomous AI agent systems, grounded in a minimal four-element ontology and organized around four invariant transition boundaries in the agentic execution loop.

The taxonomy's primary value is structural stability. Derived from the execution loop ontology rather than from an enumeration of known attacks, it does not become obsolete when adversaries introduce novel variants. A boundary remains a boundary regardless of the mechanism by which it is crossed.

The three extensions introduced here address failure modes that earlier single-boundary analysis did not adequately capture. Temporal context entropy provides a framework for reasoning about dangerous information accumulation across the loop's temporal dimension. The Canonical Execution Event provides a normalized schema for policy evaluation and audit that is consistent across frameworks and deployment configurations. The multi-agent trust propagation analysis establishes that agent-to-agent communication introduces a failure surface qualitatively distinct from single-agent boundary repetition.

The two product wedges, APG and SDL, follow directly from the boundary model. They are the natural enforcement architectures at the two major sides of the agentic security problem: what the agent can see, and what the agent can do.

The practical value of a compact ontology is that it stops scope drift. With the AFB model in place, the question becomes precise: which boundary do we own, how well do we enforce it, and what does the architecture look like when we enforce it well enough that everything else can be built on top?

## References


[1] Goodfellow, I., Shlens, J., and Szegedy, C. (2014). Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*.

[2] Carlini, N. and Wagner, D. (2017). Towards evaluating the robustness of neural networks. *IEEE Symposium on Security and Privacy*, pp. 39-57.

[3] OWASP Foundation (2025). *OWASP Top 10 for Large Language Model Applications 2025*. https://owasp.org/www-project-top-10-for-large-language-model-applications/

[4] OWASP Agentic Security Initiative (2026). *OWASP Top 10 for Agentic Applications 2026*. https://owasp.org/www-project-agentic-security/

[5] NIST (2017). *SP 800-63C: Digital Identity Guidelines, Federation and Assertions*. National Institute of Standards and Technology.

[6] Hu, V. C., Ferraiolo, D., Kuhn, R., et al. (2014). *Guide to Attribute Based Access Control (ABAC): Definition and Considerations*. NIST Special Publication 800-162.

[7] ICO (2023). *Guidance on Pseudonymisation*. Information Commissioner's Office.

[8] Perez, F. and Ribeiro, I. (2022). Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527*.

[9] Greshake, K., Abdelnabi, S., Mishra, S., et al. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*.

[10] Anthropic (2024). *Model Context Protocol Specification*. https://modelcontextprotocol.io/